

QALA

Universal Solution Factory Operating System

Unified Design & Architecture Document

Synthesised from Platform Concept, SDD v1/v2, LLD, RFC v2, PRD, Requirements, Workflows, UI/UX Spec, and Business Plan

Version 1.0 | March 2026 | Confidential

Document Type	Unified Design & Architecture Document
System	Qala Universal Solution Factory Operating System
Version	1.0
Classification	Confidential — Not for Distribution without Written Consent
Source Documents	Platform Concept v1 · SDD v1 · SDD v2 · LLD v1 · RFC v2 · PRD v1 · Requirements v1 · Workflows v2 · UI/UX Spec v1 · Business Plan v1
Audience	Engineering · Architecture · Product · Security · Leadership · Investors

1 Executive Summary

Qala is a Universal Solution Factory Operating System — a single governed platform for the end-to-end creation, management, and lifecycle governance of solutions across every domain and scale. Whether the solution is a software microservice, a pharmaceutical formulation, a financial model, an agricultural protocol, a legal product, or a tax rule set — Qala governs it through a consistent, structured, and intelligence-augmented lifecycle.

Core Thesis

Every person or organisation that builds something of value deserves world-class delivery infrastructure. Qala makes governed, reproducible, AI-augmented solution lifecycle management accessible to everyone — from a solo developer to a global enterprise.

The software and technology industry spends an estimated 30–50% of engineering effort managing the infrastructure required to build, deliver, and govern solutions — not on the solutions themselves. Qala eliminates this waste by providing a single, opinionated, extensible operating system that spans the full lifecycle: from first idea to production deployment to retirement.

1.1 The Seven Core Problems Qala Solves

Dimension	The Problem Today	Qala's Solution	Success Metric
Build Reproducibility	"Works on my machine"	Hermetic SDEs with digest-pinned environments	Zero environment CI failures
Security	Manual, late, reactive	SAST/SCA/DAST + SEM embedded at every layer	95%+ vulnerabilities in dev phase
Environment Drift	Silent divergence	Environment-as-code + continuous drift detection	All drift detected within 15 minutes
Knowledge Silos	Lives in individuals' heads	Versioned artefacts + AI semantic search	New developer productive in < 1 day
Governance	Manual, incomplete	Immutable audit trail from day one	Compliance reports in < 4 hours
AI Integration	Siloed, not workflow-integrated	AI Agent woven through every platform domain	30%+ reduction in defect escape rate
Toolchain Fragmentation	30–50% of effort on tooling	Unified Solution Factory model	40–60% reduction in toolchain overhead

1.2 Key Platform Metrics

Metric	Target / Value
Active SDEs at 12 months	10,000 target
SDE provisioning time	< 10 minutes
API P99 latency (read)	< 80ms
Platform uptime (SaaS)	99.9% (standard) / 99.95% (enterprise)
Event processing throughput	> 100,000 domain events/sec platform-wide
Security scan detection time	< 90 seconds (automated SEM)
Rollback time (Blue/Green)	< 60 seconds
Compliance audit prep time	< 4 hours (vs 4–12 weeks industry average)

2 Platform Philosophy & Design Principles

Qala is architected around a small number of binding principles that govern every design decision. Any component, service, or feature that violates a principle requires explicit architectural justification and approval.

2.1 The Root Factory Principle

Architectural Reality, Not Metaphor

Qala is itself a Solution Factory — the Root Solution Factory from which all other factories, environments, models, and solutions descend. Every platform update goes through a Change Control Request. Every release is governed. Every architectural decision is recorded as an Architecture Decision Record in the Qala root factory. The platform is self-describing and self-governing.

2.2 Binding Architectural Principles

Principle	Constraint	Implementation	Scope
P1: Event Sourcing	All state changes derived from immutable ordered event log	Event log is system of record; projections are derived	Solutions, SDEs, CCRs, releases, audit events
P2: CQRS	Command and query paths separated at service level	No synchronous reads in command handlers	All domain services
P3: Immutable Artefacts	Released artefacts and audit events are never modified	Rollback = activating a previous version, never modifying history	Artefact registry, audit vault
P4: Tenant Isolation	All data, events, secrets isolated per tenant	Row-level security + tenant-scoped encryption keys	All storage layers
P5: Composable Deployment	Every SDE is independently deployable	Platform runtime decoupled from SDE runtime	SDE Kubernetes workloads
P6: Domain-Agnostic Core	Core services are domain-agnostic	Domain-specific logic lives in Domain Packs	Platform core services
P7: API-First	Every capability exposed via versioned API before UI	UI is a consumer of the API, not a privileged client	All platform capabilities
P8: Zero-Trust Security	All service-to-service comm is mutually authenticated	mTLS throughout; no implicit trust; every request carries verifiable identity	Service mesh (Istio)
P9: Observability by Default	Every service emits structured logs, traces, and metrics	No operational state unreachable without code changes	OpenTelemetry + Prometheus

Principle	Constraint	Implementation	Scope
P10: Graceful Degradation	Platform remains partially functional during failures	No single-service failure causes total platform unavailability	Circuit breakers, fallbacks

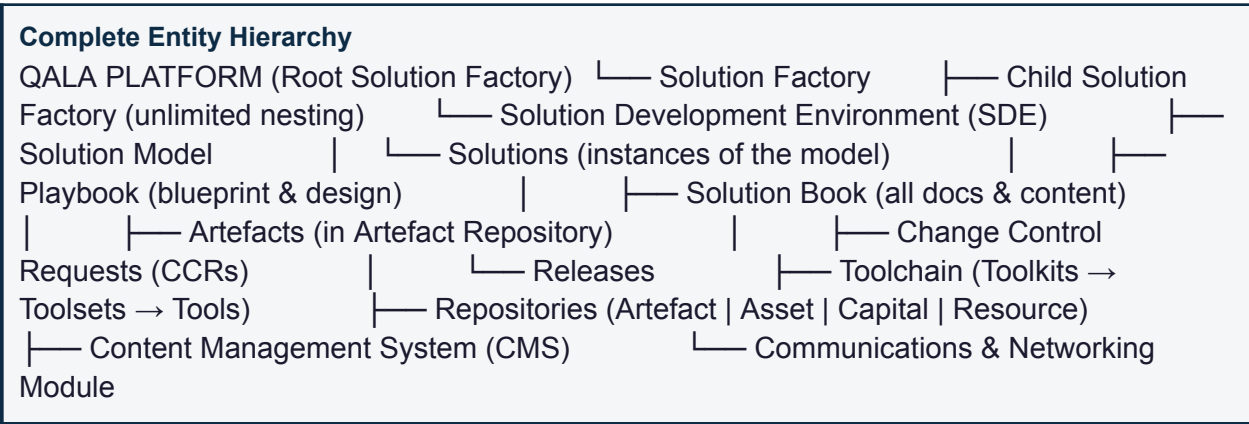
2.3 Platform Design Philosophy

Principle	Description	Implementation Note
Universal by Design	One platform for every solution type	Personal to enterprise. Software to physical. Digital to regulatory.
Hierarchical Composition	Factories produce factories; solutions nest inside solutions	Hierarchy is unlimited in depth and breadth.
Governance as Foundation	Change control and audit trails are the substrate	Not optional features — core to every entity.
Distributable & Deployable	SDEs are packaged, versioned, and distributable	Any target: local, cloud, edge, or offline.
Composable Everything	Every entity is composable from reusable components	No lock-in. No silos.
Root Factory Principle	Qala can produce instances of itself	Tiered, hierarchical, self-similar at every scale.

3 Core Entity Model & Hierarchy

Every entity in Qala is either a solution or produces solutions. The platform is structured as a strict containment hierarchy with seven levels of depth. This section defines the canonical entity model that every subsystem must conform to.

3.1 Entity Hierarchy



3.2 Entity Summary Table

Entity	Layer	Produced By	Produces	Governed By
Qala Platform	Root	N/A (the root)	All entity types	Qala platform governance
Solution Factory	Organizational	Qala Platform or Parent Factory	Child Factories, SDEs	Factory policies, RBAC
Solution Dev Env (SDE)	Workspace	Solution Factory	Solutions, Artefacts, Content	SDE config, factory policies
Solution Model	Schema/Blueprint	Factory Admin / Domain Team	Solution instances	Model versioning, domain pack
Domain Pack	Schema Extension	Qala or custom authors	Domain-specific field extensions	Pack versioning, factory approval
Solution	Output	SDE (developer action)	Artefacts, Releases, Change history	CCR, lifecycle state machine, model
Playbook	Documentation	SDE member or AI Agent	Design decisions, ADRs, blueprints	Solution versioning, CMS
Solution Book	Documentation	Auto-generated + SDE members	Comprehensive solution documentation	Solution versioning, CMS, access policy
CCR	Governance	Any authorized user	Approved changes, compliance evidence	Risk routing, approval chain policy

Entity	Layer	Produced By	Produces	Governed By
Release	Deployment	Release Manager	Deployed solution, release artefacts	Release policy, quality gates
Artefact	Output	SDE build/toolchain	Deployable packages, evidence records	Artefact repository policies
Toolchain	Infrastructure	Qala Marketplace or custom	Built artefacts, test results	Toolchain versioning, SDE config
AI Agent	Intelligence	Qala platform (system)	Recommendations, risk scores, insights	AI policy, privacy settings
Audit Event	Compliance	Every governed action (automatic)	Compliance evidence packages	Audit vault (immutable)

3.3 Solution Structure Model — Seven-Level Hierarchy

Qala enforces a strict seven-level hierarchy that governs how every solution is decomposed and described. This hierarchy applies universally regardless of domain.

Level	Element & Definition
1	System — Top-level organisational unit grouping applications that fulfil a coherent purpose
2	Application — Deployable unit of functionality residing within a System
3	Process — Discrete unit of work executed within an Application
4	Component — Bounded, reusable building block within a Process
5	Interface — Contract surface of a Component; declares Imports and Exports
6	Message — Unit of communication flowing through an Interface (Event or State type)
7	Data Structure — Schema of a Message payload, composed of typed fields

3.4 Six First-Class Solution Types

Solution Type	Description
Application	A software application delivered to end-users or integrated systems
System	A coordinated collection of applications serving a unified purpose
Good	A tangible, physical, or digital deliverable produced by the factory
Product	A commercially packaged, versioned, and distributable artefact
Service	An ongoing capability delivered to consumers on a continuous basis
Platform	A foundation on which other solutions are built and operated

4 Solution Factory

A Solution Factory is the primary organizational unit in Qala. It is a governed namespace that produces and manages Solution Development Environments, child factories, and all solution activity within a defined scope.

4.1 Factory Types & Tiers

Factory Type	Intended Use	Key Characteristics	Scale
Personal Factory	Single user, personal projects	Single owner, unlimited SDEs, private by default	Solo
Team Factory	Small group building products	Multi-member, role-based access, shared templates	2–50 users
Organization Factory	Business managing multiple products	Org-level RBAC, department sub-factories, compliance	51–5,000 users
Enterprise Factory	Large enterprise, complex governance	Hierarchical sub-factories, SSO, data residency, SLA	5,000+ users
Platform Factory	Produces reusable factory templates	Public/gated, versioned factory blueprints, marketplace-publishable	Varies
Root Factory (Qala)	The Qala platform itself	System-level; produces all factory types; all domain packs	Global

4.2 Factory Governance

Each factory defines its own governance policy, inherited (and optionally overridden) by child factories. Governance configuration includes:

- Change Control Policy — whether CCRs are required, routing rules, approval thresholds by risk level
- Release Policy — which quality gates must pass before a solution can be released
- Version Strategy — SemVer, CalVer, or custom versioning scheme
- Audit Policy — what events are logged, retention period, export schedule
- Access Control — RBAC roles defined at factory level; inheritable by child factories and SDEs
- Domain Pack — the domain-specific schema extension active in this factory

4.3 Factory as a Product

A configured factory — with its templates, domain packs, toolchains, and policies — can be packaged as a Factory Blueprint and shared through the Qala Marketplace. Organisations can

publish blueprints for their industry (e.g. "GxP-Compliant Pharma Factory Blueprint") which other organisations can instantiate and customise. This creates a marketplace of governed, pre-configured factory patterns.

5 Solution Development Environment (SDE)

The Solution Development Environment is the foundational deployable unit of Qala. It is a self-contained, configurable, distributable workspace that defines all conditions for creating and operating solutions. An SDE is not a static folder — it is a living, governed, versioned environment with its own lifecycle.

5.1 SDE Core Properties

Property	Description	Examples
Deployable	Can be deployed to any target: cloud platform, logical environment designation, or physical machine	SaaS, cloud, on-premises, edge, air-gapped
Configurable	All aspects parameterised and adjustable without structural change	Environment variables, language runtimes, tool versions, profiles
Distributable	Packaged, shared, replicated, and instantiated in new contexts	Consistent environments across teams, regions, and instances

The SDE Analogy

The SDE is to Qala what a kernel is to an operating system: it provides the managed substrate on which solutions live, without being the solution itself.

5.2 SDE Lifecycle States

State	Description	Entry Condition	Exit Condition
Draft	SDE being configured; not yet active	SDE created	Configuration complete + activated
Active	SDE live and in productive use	Activation confirmed	Manual suspend / policy trigger
Snapshotted	Point-in-time capture taken; SDE continues running	Snapshot command issued	Continuous; SDE remains Active
Suspended	SDE paused; resources retained; reactivatable	Manual or policy trigger	Manual reactivation or decommission
Rolled Back	SDE restored to a prior snapshot state	Rollback command issued	Continues as Active from prior snapshot

State	Description	Entry Condition	Exit Condition
Quarantined	SDE isolated by SEM on threat detection	SEM threat confirmation	Clean security scan + clearance
Archived	Work complete; read-only; artefacts preserved	All solutions released or migrated	Cannot be reactivated (clone to new)
Decommissioned	Permanently shut down; resources released	Archive + retention elapsed + approval	Permanent deletion

5.3 SDE Composition

An SDE is composed of the following constituent elements, each independently configurable and version-controlled:

Element	Description
Environment Variables	Runtime configuration values injected into the environment
Language Runtimes	Supported programming languages, runtimes, and compilers with pinned versions
Toolchain	Attached toolkits, toolsets, toolchains, and third-party integrations
Configuration Files	System-level and service-level configuration files; the .qala.yaml manifest
Profiles	Named configurations for different deployment targets or roles
Parameters & Options	Tuneable values and feature flags modifying SDE behaviour
Repositories	Four built-in types: Artefact, Asset, Capital, Resource
CMS	Integrated content management for documentation, media, and structured data
Communications Module	Messaging, webhooks, event streams, API gateway config, notification rules
Scalability Config	Min/max instances, auto-scale policies, multi-region config, resource limits

5.4 Toolchain Hierarchy

Level	Definition & Examples
Tool	A single executable capability (linter, test runner, code generator, data processor)
Toolset	A coherent collection of tools for a specific task (e.g. Testing Toolset: pytest + coverage + mock)
Toolkit	A complete toolbox for a development domain (e.g. Python Backend Toolkit)
Toolchain	Orchestrated sequence of toolkits producing a governed pipeline: develop → build → test → gate → release

5.5 The .qala.yaml SDE Manifest

SDE Manifest Format

Every SDE contains a .qala.yaml manifest at its root:

```
sde: id: sde-a3f9c2 name:
api-gateway-sde version: 2.4.1 domain: software factory: acme-platform-teamtoolchain: -
id: node-20-lts - id: eslint-8 - id: qala-ci-pipeline
repositories: artifacts:
repo-artifacts-sde-a3f9c2 assets: repo-assets-sde-a3f9c2scalability: min_instances: 1
max_instances: 10 auto_scale_policy: cpu_70pct
```

5.6 SDE Repository Types

Repository Type	Contents	Key Capabilities
Artefact Repository	Built outputs: binaries, packages, container images, test results, release bundles	Push/pull, versioned, SHA-256 integrity, SLSA attestation, retention policy
Asset Repository	Source inputs and design assets: source code, design files, schemas, raw media	Version-controlled, diff-capable, linked to SDE version history
Capital Repository	Intellectual and financial capital: IP, patents, licences, proprietary datasets	Access-controlled, immutable records, chain-of-custody
Resource Repository	Infrastructure resources: IaC files, capacity reservations, cloud resources	Scoped secrets, IaC execution context, resource graph

6 Solution Model, Domain Packs & Universal Schema

A Solution Model is the structured definition, schema, and behavioral specification that governs how a specific type of solution is created, validated, and managed. Models are to solutions what blueprints are to buildings — they define the pattern, not the instance.

6.1 Universal Solution Schema

Every solution model shares a universal core schema. Domain Packs add fields on top of this core without replacing it:

Field Group	Fields (always present)
Identity	id, name, slug, version, type, domain, tags, labels
Ownership	owner_id, created_by, last_modified_by, assigned_to
Lifecycle	state, state_history, previous_state, transition_reason, transition_timestamp
Versioning	version, version_history, version_notes, changelog, is_latest, is_stable, is_deprecated
Associations	parent_solution_id, child_solution_ids, related_solution_ids, sde_id, factory_id, model_id
Quality	quality_gate_results, open_defects, defect_summary, last_gate_evaluation
Change Control	open_ccr_count, ccr_ids, last_ccr_id, change_freeze_until
Artefacts	artefact_ids, primary_artefact_id, artefact_registry_ref
Documentation	playbook_id, solution_book_id, readme_url, doc_version
Compliance	compliance_status, regulatory_declarations, evidence_package_ids, last_audit_date
AI	ai_recommendations, last_ai_analysis, risk_score, drift_status

6.2 Domain Packs

A Domain Pack is a versioned JSON Schema extension that adds domain-specific fields, validation rules, lifecycle states, and quality gate definitions to the Universal Solution Schema. Domain Packs are the mechanism by which Qala adapts to any industry without changing the platform core.

Domain Pack	Example Fields Added	Quality Gate Types
Software	Language, runtime, test coverage, SAST results	SAST, Coverage, Dependency Check, SLSA
CPG / Pharma	INCI names, batch info, regulatory filings, stability data	Batch Release Gate, QP Sign-off, Regulatory Pre-check
Financial	Strategy parameters, risk metrics, regulatory filings, mandates	Risk Gate, Drawdown Gate, Mandate Compliance Gate
Agricultural	Crop data, field GPS, treatment protocols, yield targets	Agronomic Gate, Traceability Gate, Seasonal Gate
Legal / Tax	Jurisdiction, effective dates, legislative reference, filing status	Legal Review Gate, Compliance Gate, Effective-Date Gate
Research	Hypothesis, experimental design, dataset versions, IRB approval	Peer Review Gate, Reproducibility Gate, IRB Gate

6.3 Solution Lifecycle States (Universal Base)

State	Description	Key Actions
Conception	Idea captured; not yet in development	Define scope, attach model, assign SDE, create playbook
Draft	Active development; being designed and built	Edit, commit, test, iterate, create CCRs
In Review	Development complete; undergoing evaluation	Submit for review, quality gates, approvals
Approved	Review passed; authorised for release	Create release, generate artefacts, prepare package
Released	Packaged, attested, available for deployment	Deploy, distribute, install, rollback-capable
Live / Active	Deployed and operational in target environment	Monitor, observe, receive CCRs for operational changes
Deprecated	End-of-life; still operational but flagged	Notify consumers, create migration CCR, set retirement date
Retired	No longer operational; fully decommissioned	Read-only history; archive artefacts

7 Change Control & Versioning

Every intentional change to any solution, SDE, model, or factory in Qala is a governed event. Change Control Requests (CCRs) are the mechanism through which changes are proposed, evaluated, approved, and executed. The CCR model is universal — it governs software dependency updates with the same rigor as pharmaceutical ingredient substitutions or financial model recalibrations.

7.1 Change Control Request (CCR)

CCR Attribute	Description
Identity	CCR ID, title, type, status, priority, risk score
Scope	Affected solutions, SDEs, and factory refs. Impact analysis (cascading effects).
Content	Change description, justification, implementation plan, rollback plan, regulatory implications.
Risk	Computed risk score (1–10): change type, impact scope, regulatory dimension, domain risk factors.
Routing	Approval chain auto-computed from risk level, solution type, factory CCR policy, domain rules.
Evidence	Supporting artefacts: test results, simulation outputs, regulatory pre-checks, expert reviews.
Status	Draft → Submitted → In Review → Approved / Rejected / Deferred → Implemented → Verified
Audit	Every status change recorded with actor, timestamp, and reason. Immutable audit trail.

7.2 CCR Risk Scoring & Approval Routing

Risk Band	Score Range	Approval Required	Examples
Low	1–3	Solution owner or auto-approve if policy allows	Updating README, minor dependency bump, config option
Medium	4–6	Two-stage: Technical lead + domain expert	Feature affecting 5–10 solutions, new integration
High	7–8	Three-stage: Tech lead + domain expert + compliance	Regulatory-relevant change, cross-domain impact, ingredient substitution

Risk Band	Score Range	Approval Required	Examples
Critical	9–10	Full Change Control Board (CCB). May require external sign-off.	Recall initiation, emergency change affecting 50+ solutions, structural model change

7.3 CCR Types

CCR Type	When Used	Auto-Applies To
Feature Addition	Adding new capability to a solution	All solution types
Defect Fix	Correcting an error, bug, or non-conformance	All solution types
Configuration Change	Modifying settings, parameters, or environment configuration	All solution types
Dependency Update	Updating a referenced tool, library, integration, or service	Software, Platform, Research
Ingredient Substitution	Replacing a component material or input substance	CPG, Pharma, Agricultural
Mandate Amendment	Modifying the governing mandate, policy, or scope	Financial, Legal, Service, Tax
Legislative Update	Updating a solution to reflect a change in law or regulation	Tax, Legal, Compliance, Financial
Security Patch	Addressing a security vulnerability or threat	Software, Platform, Toolchain
Emergency Change	Urgent unplanned change bypassing standard routing; post-facto review required	All solution types

7.4 Versioning Strategy

Versioning applies to every entity: solutions, SDEs, solution models, factories, playbooks, toolchains, and domain packs.

Strategy	Format	Use Case
Semantic Versioning (SemVer)	MAJOR.MINOR.PATCH (e.g. 2.4.1)	Software, platform, and library solutions
Calendar Versioning (CalVer)	YYYY.MM.DD (e.g. 2025.03.15)	Tax rules, regulatory solutions, compliance frameworks

Strategy	Format	Use Case
Sequential Versioning	Incrementing integers (e.g. v42)	Batch records, experiment runs, iteration-heavy research

Immutable Version Principle

Once a version is released, it is immutable. It cannot be deleted, edited, or replaced. A new change creates a new version. This ensures every released artefact has a permanent, auditable record. Rollback means activating a previous immutable version — never modifying history.

8 System Architecture

Qala is implemented as a polyglot microservices platform organised into four architectural planes, each implemented in the language best suited to its operational characteristics. The Kernel provides system-level coordination; the Control Plane manages all API-facing operations; the Execution Plane handles async and compute-intensive work; the Intelligence & Security Plane provides AI and security capabilities.

8.1 Architectural Planes

Plane	Language	Ports	Responsibilities
Kernel System	Rust	:7000	Service registry, subsystem health monitoring, command bus, event aggregation
Control Plane	Go	:8080–8083, :8085	API gateway, user identity, SDE lifecycle, workspace/CMS, artefact management
Execution Plane	Scala	:8084, :8086, :8089	CI/CD workflow engine, data platform and analytics, notification dispatch
Intelligence & Security	Rust	:8087, :8088	AI recommendations/analysis, security event management, threat detection

8.2 Complete Service Map

Service	Language	Port	Responsibilities
qala-kernel	Rust	7000	Registry, Health, Command Bus, Event Aggregation
api-gateway	Go	8080	API ingress + service routing, auth middleware, rate limiting
user-identity-service	Go	8081	User CRUD, authentication, RBAC seed, identity management
sde-management-service	Go	8082	SDE lifecycle, snapshot/rollback, solutions, solution factories
workspace-cms-service	Go	8083	Workspaces, file content management, CMS events
workflow-ci-cd-service	Scala	8084	Pipeline run/status, BUILD_EVENTS, CI/CD orchestration
artifact-package-service	Go	8085	Artefact metadata, upload/download, ARTIFACT_EVENTS
data-platform-service	Scala	8086	Analytics pipelines, metrics, DATA_EVENTS

Service	Language	Port	Responsibilities
ai-agents-service	Rust	8087	Recommendations, SDE analysis, AI_RECOMMENDATIONS
security-sem-service	Rust	8088	Threat detection, policy enforcement, SECURITY_EVENTS
notifications-service	Scala	8089	Message dispatch, multi-channel routing, NOTIFICATIONS

8.3 Kernel Subsystems

Subsystem	Responsibility	Key Endpoints
Registry	Tracks all registered services, metadata, and heartbeat/health status	POST /v1/kernel/register-service, GET /v1/kernel/status
CommandBus	Processes kernel-level commands: set_subsystem_status, set_service_status	POST /v1/kernel/command
EventAggregator	Counts, classifies, and stores recent events per topic; real-time event summary	POST /v1/kernel/events, GET /v1/kernel/events

8.4 Technology Stack

Layer	Technology	Rationale
Backend Services	Go 1.22 (primary), Python 3.12 (ML/AI)	Performance, low memory, excellent concurrency; Python ML ecosystem
API Gateway	Kong Gateway + custom Go middleware	Production-proven, plugin ecosystem, rate limiting, JWT validation
Message Broker	Apache Kafka 3.7 (KRaft mode)	Persistent event log, at-least-once delivery, compaction for projection rebuilds
Primary Database	PostgreSQL 16 with Citus	ACID, row-level security for tenant isolation, JSONB for schema-flexible fields
Object Storage	S3-compatible (AWS S3 / Cloudflare R2 / MinIO)	Artefact storage, immutable versioned buckets, pre-signed URL access
Search	OpenSearch 2.x	Full-text and faceted search over solutions, playbooks, and solution books

Layer	Technology	Rationale
Cache	Redis 7 (cluster mode)	Read projection cache, distributed rate-limit counters, session store
Secrets	HashiCorp Vault / AWS Secrets Manager	Dynamic secrets, per-tenant encryption keys, automated rotation
Service Mesh	Istio 1.21 (mTLS, traffic management)	Zero-trust service-to-service auth, circuit breaking, traffic shaping
Container Runtime	Kubernetes 1.30 (EKS / AKS / GKE / bare metal)	SDE runtime, factory namespace isolation via k8s namespaces + network policies
Observability	OpenTelemetry + Prometheus + Grafana + Loki	Unified trace/metric/log pipeline, vendor-neutral, exportable
Frontend	React 19 + TypeScript + TanStack Query + Zustand	Component-based, type-safe, optimistic UI, real-time via WebSocket
Mobile	React Native (Expo SDK 52)	Cross-platform (iOS + Android), shared business logic with web
CLI	Go (Cobra framework) — ships as qala CLI	Single binary, cross-platform, auto-update, shell completion

9 Data Model & Event Sourcing

Qala uses event sourcing as its primary persistence strategy. Every domain entity is represented as an ordered sequence of immutable events. Current state is a projection derived by replaying events. This section defines the canonical event schemas, aggregate boundaries, projection models, and storage layout.

9.1 Aggregate Boundaries

Aggregate	Root Entity	Commands Accepted
FactoryAggregate	SolutionFactory	CreateFactory, UpdatePolicy, ArchiveFactory, AddMember, AttachDomainPack
SDEAggregate	SDE	CreateSDE, ActivateSDE, SuspendSDE, UpdateConfig, AttachToolchain, ScaleSDE
SolutionAggregate	Solution	CreateSolution, TransitionState, RollbackToVersion, AttachPlaybook, AttachSolutionBook
CCRAggregate	ChangeControlRequest	SubmitCCR, ApproveCCR, RejectCCR, DeferCCR, WithdrawCCR, ImplementCCR
ReleaseAggregate	Release	CreateRelease, AddArtefact, ApproveRelease, PublishRelease, RecallRelease
IdentityAggregate	User / ServiceAccount	CreateUser, AssignRole, RevokeRole, EnableMFA, SuspendUser, CreateServiceAccount
ModelAggregate	SolutionModel	CreateModel, PublishModel, DeprecateModel, UpdateLifecycle, AttachDomainPack

9.2 Canonical Event Envelope

Event Envelope Schema (JSON)

```
{ "event_id":      "uuid-v7",           // Time-ordered UUID (sortable) "event_type":
  "solution.state_transitioned", // domain.action format "event_version": "1",           //
  Schema version for this event type "aggregate_id": "uuid", "aggregate_type":
  "SolutionAggregate", "tenant_id":      "uuid",           // Tenant scope (for isolation)
```

```

"factory_id": "uuid", "correlation_id": "uuid", // Ties related events "causation_id":
"uuid", // ID of command that caused this event "actor_id": "uuid", "timestamp":
"2026-03-09T12:00:00.000Z", "schema_version": "qala/v1", "payload": { ... } //
Event-type specific data}

```

9.3 Event Topics

Topic	Publisher / Consumer	Retention	Key Event Types
USER_EVENTS	Identity service → SDE, Workspace, Notification	30 days	USER_CREATED, USER_UPDATED, ROLE_ASSIGNED
SDE_EVENTS	SDE Management → AI, SEM, Notification, Kernel	30 days	SDE_CREATED, SNAPSHOTTED, QUARANTINED, ARCHIVED
CMS_EVENTS	Workspace/CMS → Notification, Analytics	30 days	WORKSPACE_CREATED, CONTENT_UPDATED
BUILD_EVENTS	CI/CD → Artefact, Notification, AI	90 days	BUILD_STARTED, BUILD_PASSED, ATTESTATION_CREATED
ARTIFACT_EVENTS	Artefact → Data Platform, Notification	90 days	ARTIFACT_PUBLISHED, ARTIFACT_RECALLED
DATA_EVENTS	Data Platform → Analytics, Notification	90 days	PIPELINE_RUN, ANALYTICS_UPDATED
SECURITY_EVENTS	SEM → Notification, SDE Mgmt, Kernel	1 year	THREAT_DETECTED, QUARANTINED, VULNERABILITY_FOUND
COMPLIANCE_EVENTS	All services → Audit Vault	7 years	POLICY_CHECKED, VIOLATION_DETECTED, EVIDENCE_GENERATED
NOTIFICATIONS	All services → Notification service	14 days	Routing and dispatch events
AI_RECOMMENDATIONS	AI Agents → SDE Mgmt, User-facing	14 days	RECOMMENDATION_PUBLISHED, PREDICTION_ISSUED
AUDIT_EVENTS	Every state-modifying action (automatic)	7 years	Every governed action — append-only, cryptographically chained

9.4 Core Database Schema

All tables use row-level security (RLS) policies to enforce tenant isolation at the database layer. The events table is append-only. Projection tables are derived from events and rebuilt by replaying the event log.

events table — write side (append-only)

```
CREATE TABLE events ( event_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
event_type    TEXT NOT NULL, event_version SMALLINT NOT NULL DEFAULT 1,
aggregate_id  UUID NOT NULL, aggregate_type TEXT NOT NULL, tenant_id    UUID
NOT NULL, factory_id    UUID, correlation_id UUID, causation_id  UUID, actor_id
UUID NOT NULL, occurred_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(), payload
JSONB NOT NULL, metadata    JSONB DEFAULT '{}');-- RLS: SELECT WHERE tenant_id
= current_setting('qala.tenant_id')::uuid
```


10 API Design & Contracts

Qala exposes three API surfaces: a REST API for CRUD and command operations, a GraphQL API for flexible querying, and a WebSocket API for real-time event subscriptions. All surfaces are versioned, authenticated, and rate-limited at the API Gateway.

10.1 API Versioning Strategy

API Surface	Version Scheme	Current Version	Sunset Policy
REST API	URL path: /api/v1/...	v1 (stable), v2 (beta)	18 months minimum post-supersession
GraphQL API	Schema version in SDL header	Schema v1	@deprecated fields supported 12 months
WebSocket API	Event envelope schema_version	qala/v1	Old schema supported 6 months post-new-version
gRPC (internal)	Protobuf package version	qala.v1.*	Internal only; 90-day sunset on breaking changes

10.2 Complete API Endpoint Catalogue

Gateway (:8080)

Endpoint	Description
GET /health	Platform health check
GET /routes	List all registered routes
ANY /api/*	Proxy to downstream services

User Identity (:8081)

Endpoint	Description
POST /users	Create a new user account
GET /users	List all users (with filtering)
GET /users/{id}	Retrieve user profile and metadata
PUT /users/{id}	Update user profile
DELETE /users/{id}	Deactivate and remove a user account

SDE Management (:8082)

Endpoint	Description
POST /sdes	Provision a new SDE
GET /sdes	List all SDEs (with filtering)
GET /sdes/{id}	Retrieve SDE configuration and status
PATCH /sdes/{id}	Update SDE configuration
DELETE /sdes/{id}	Decommission an SDE
POST /sdes/{id}/snapshot	Take a point-in-time snapshot of an SDE
POST /sdes/{id}/rollback	Restore an SDE to a prior snapshot
POST /sdes/{id}/solutions	Associate a solution with an SDE
DELETE /sdes/{id}/solutions/{sid}	Remove a solution association from an SDE
POST /solutions	Create a new solution record
GET /solutions	List all solutions (with filtering)
GET /solutions/{id}	Retrieve a specific solution
PATCH /solutions/{id}	Update solution metadata
DELETE /solutions/{id}	Archive or delete a solution
POST /factories	Create a new Solution Factory
GET /factories	List all Solution Factories
GET /factories/{id}	Retrieve a specific Factory
PATCH /factories/{id}	Update Factory configuration
DELETE /factories/{id}	Dissolve a Solution Factory
POST /factories/{id}/sdes	Add an SDE to a Factory
DELETE /factories/{id}/sdes/{sdeld}	Remove an SDE from a Factory

Workspace CMS (:8083), CI/CD (:8084), Artefacts (:8085)

Endpoint	Description
POST /workspaces	Create a new workspace
GET /workspaces/{id}	Retrieve workspace metadata
PUT /workspaces/{id}	Update workspace configuration

Endpoint	Description
DELETE /workspaces/{id}	Delete a workspace
POST /workspaces/{id}/content	Upload or update content within a workspace
GET /workspaces/{id}/content/{filePath}	Retrieve specific content file
POST /pipelines/run	Trigger a CI/CD pipeline
GET /pipelines/{id}/status	Query pipeline execution status
POST /artifacts/upload	Upload an artefact to the registry
GET /artifacts	List all artefacts
GET /artifacts/{id}	Retrieve artefact metadata
GET /artifacts/{id}/download	Download an artefact binary
DELETE /artifacts/{id}	Remove an artefact

AI Agents (:8087), Security SEM (:8088), Notifications (:8089), Data Platform (:8086), Kernel (:7000)

Endpoint	Description
GET /recommendations	Retrieve AI-generated recommendations
POST /analyze_sde	Request an AI analysis of a specific SDE
GET /threats	List active and historical security threats
POST /policy/update	Push a new security policy to the SEM
POST /scan_sde	Initiate a security scan on an SDE
POST /notify	Dispatch a notification
GET /notifications	List all notifications
GET /notifications/{id}	Retrieve a notification record
POST /data/pipeline/run	Trigger a data analytics pipeline
GET /data/analytics	Query analytics output
GET /v1/kernel/status	Get kernel and subsystem status
POST /v1/kernel/register-service	Register a service with the kernel
POST /v1/kernel/events	Publish events to the kernel
GET /v1/kernel/events	Get aggregated event counts by topic

Endpoint	Description
POST /v1/kernel/command	Execute a kernel command

11 Security Model

Qala is built on a zero-trust security architecture. Every service-to-service communication is mutually authenticated. Every user request carries a verifiable identity. Security is not a bolt-on — it is embedded at every layer of the platform.

11.1 Authentication & Authorisation

Credential Type	Used By	Mechanism	Scope
User JWT	Web app, mobile, CLI (user sessions)	JWT RS256, signed by Identity Service; embedded: user_id, tenant_id, roles, factory_scopes	1h access token, 30d refresh token
Service Account Token	CI/CD integrations, automation scripts	Opaque token mapped to service account identity; scoped to one factory + SDE list	No expiry by default; rotate on policy
M2M Client Credentials	Backend service-to-service (OAuth 2.0)	Client credentials grant; JWT-based; scoped to specific service operations	15 minutes; auto-refreshed
API Key (legacy)	Simple integrations, webhooks	Hashed API key stored per tenant; checked against HMAC; deprecated — use service accounts	Never expires unless revoked; limit 10/tenant

11.2 Security Event Management (SEM)

The SEM subsystem provides continuous, active security enforcement across all SDEs and Factories:

Capability	Description
Threat Detection	Continuous analysis of SDE behaviour, network traffic, and configuration state for anomalies
Automatic Quarantine	Confirmed threat detection triggers automatic SDE isolation within 90 seconds
Policy Enforcement	Security policies defined centrally and pushed to all SDEs; violations generate SECURITY_EVENTS
Secrets Vault	All credentials stored encrypted; no secrets in plain text; automated rotation
Post-Quarantine Recovery	Clean scan required + explicit Security Engineer clearance before SDE returns to Active

Capability	Description
Cross-SDE Evaluation	On quarantine, all other SDEs in the same factory evaluated for indicators of compromise
Audit Trail	All security actions immutably logged with actor, timestamp, event type, affected resource
Vulnerability SLAs	Critical: remediation within 24h; High: 72h; Medium: 30 days

11.3 RBAC — Platform Roles

Role	Scope	Key Permissions
Root Admin	Qala platform	Full system access; manage all tenants, factories, and platform settings
Factory Admin	One factory + children	Create/manage SDEs, define policies, invite members, manage domain packs, view all audit trails
Factory Member	One factory	Access to assigned SDEs; create solutions, submit CCRs, create releases (per role)
SDE Owner	One SDE	Full control of SDE: configuration, toolchains, members, repositories
Solution Developer	One or more SDEs	Create and modify solutions, commit changes, run builds, submit CCRs
Reviewer	One or more SDEs	Review and approve/reject CCRs and releases; cannot create solutions
Release Manager	One or more factories	Approve and trigger releases; manage quality gate evaluations
Compliance Officer	One or more factories	View all compliance evidence, generate audit packages, full audit trail; read-only on solutions
Guest / Reader	Specified scope	Read-only access to specified solutions, SDEs, or factories
AI Agent	System-level	Read all governed data; write AI recommendations; cannot execute state transitions

11.4 Data Security

- All data encrypted at rest using AES-256
- All data in transit encrypted using TLS 1.3 or higher
- Service-to-service communication uses mTLS within the platform perimeter
- Database access restricted to service accounts with least-privilege permissions
- Tenant isolation enforced at database layer via row-level security (RLS) policies
- All tenant data processable within designated geographic region; cross-region transfers are explicit, opt-in, and auditable

12 AI Agent & Intelligence Layer

The Qala AI Agent is not a standalone tool or chatbot. It is a persistent, context-aware intelligence layer woven through every platform domain. It operates continuously across a customer's entire solution estate, with full context of every solution's history, quality trajectory, security posture, relationship graph, domain rules, and current state.

12.1 AI Agent Capabilities

Hook	Description	Domain
ai_optimize_sde_config	Analyses SDE configuration; recommends optimisations for performance and resource efficiency	All SDEs
ai_predict_bottleneck	Uses historical data to predict upcoming performance or capacity bottlenecks	Software, Platform
ai_suggest_folder_structure	Recommends solution directory structure based on type and language profile	Software SDEs
anomaly_detection	Identifies anomalous patterns in SDE metrics, build outputs, or runtime behaviour	All SDEs
resource_prediction	Forecasts future resource consumption based on solution growth trends	All SDEs
test_generation	Generates candidate test cases for new code paths; presented for QA review	Software
defect_prediction	Analyses code changes against historical defect patterns; flags elevated-risk functions	Software
batch_quality_prediction	Predicts batch quality failure risk from formulation and process parameters	CPG / Pharma
model_drift_detection	Detects financial model drift from benchmark performance or mandate bounds	Financial
yield_prediction	Forecasts crop yield from sensor data, weather, and historical patterns	Agricultural
ccr_documentation_drafting	Auto-drafts CCR justification and impact sections from solution context	All domains
compliance_gap_detection	Identifies compliance gaps from regulatory changes against current solution state	All regulated domains

12.2 Recommendation Lifecycle

Recommendations generated by the AI Agents are published to the AI_RECOMMENDATIONS Kafka topic and surfaced through the /recommendations API endpoint. Users can accept, dismiss, or defer recommendations. Accepted recommendations are applied directly to SDE configuration

where automation is supported. All actions — including dismissals — are recorded for model feedback improvement.

AI as Companion, Not Gatekeeper

The AI Agent surfaces recommendations inline — dismissable, actionable, contextual. It never blocks a primary workflow. In regulated domains, AI is advisory; human approval is always the final gate. The AI appears when useful, disappears when not.

13 Build System, CI/CD & Release Management

Qala's build system is hermetic by design. Every build executes in a fully isolated container with no access to host network, filesystem, or ambient credentials beyond explicitly declared inputs. Every build produces a signed SLSA attestation. No artefact lacking a valid attestation can be promoted to production.

13.1 Hermetic Build Requirements

Requirement	Specification
Full isolation	Every build executes in a fully isolated container; no ambient credentials or host access
Digest-pinned dependencies	All dependencies pinned to exact version with SHA-256 digest; floating versions rejected
Digest verification	SHA-256 of every fetched dependency verified before use; mismatch fails build + SECURITY_EVENT
Bit-for-bit reproducibility	Identical source commit + environment produces identical outputs across repeated executions
SLSA attestation	Every successful build produces signed attestation: source commit SHA, env hash, dependency digests, builder identity, outputs hash
Attestation gating	Artefacts lacking valid attestation are blocked from promotion beyond the build stage
CCR for env changes	Build environment definition changes require an approved CCR before activation

13.2 CI/CD Pipeline Stages

Every CI pipeline executes the following mandatory stages in order:

Stage	Description
1. Source Validation	Verify source commit integrity; validate branch policy
2. Hermetic Environment Provisioning	Provision isolated build container from locked definition
3. Dependency Fetch + Digest Verification	Fetch all declared dependencies; verify SHA-256 digests

Stage	Description
4. SAST	Static application security testing; Critical/High findings fail pipeline
5. Build	Compile, link, package in hermetic container
6. Unit Tests	Execute unit test suite; coverage below threshold fails pipeline
7. SCA	Software composition analysis; Critical/High CVEs fail pipeline + create security bug
8. Integration Tests	Execute integration tests in ephemeral test environment
9. Container Scan	Scan container image for OS-level vulnerabilities (if applicable)
10. SLSA Attestation	Generate and sign build provenance attestation
11. Artefact Publishing	Store artefacts in Artefact Registry with SHA-256 digests
12. Quality Gate Evaluation	Evaluate combined quality gate results; publish BUILD_EVENTS

13.3 Mandatory Release Gates

Gate	Pass Criterion
Test Gate	All required test types passed (unit, integration, system, acceptance)
Coverage Gate	Code coverage above configured minimum threshold
Benchmark Gate	Performance benchmarks within acceptable deviation from baseline
Security Gate	SAST passed, no Critical/High CVEs, container scan clean
Bug Gate	No open Critical or High severity bugs in release scope (unless risk acceptance recorded)
Attestation Gate	All release artefacts carry valid, verifiable SLSA attestations
Change Control Gate	All configuration changes in scope reference approved CCRs
Approval Gate	All required sign-offs from named approvers and roles collected

13.4 Deployment Strategies

Strategy	Mechanism	Rollback Method	Best For
Blue/Green	Two environments maintained; traffic switched atomically on approval	Instant rollback by reversing traffic switch without redeployment	High-risk releases where instant rollback is essential
Canary	New version receives configurable initial traffic %; increments on health thresholds	Automatic rollback when monitored metric breaches threshold during observation window	Platform services; features requiring gradual validation
Rolling	Deploy across instance fleet with configurable batch size and health check interval	Pause rolling deployment; roll back already-updated instances	Stateless services with large instance fleets
Feature Flag	Code deployed dark; enabled selectively via configuration without redeployment	Toggle feature flag off; no redeployment required	Gradual feature rollouts; A/B testing; dark launches

14 Compliance & Audit Architecture

Qala turns compliance from a periodic fire drill into a continuous operational state. Compliance evidence is generated as a byproduct of every governed action on the platform. When an auditor arrives, Qala generates a complete, time-stamped, cryptographically signed evidence package. Design partner data shows a 94% reduction in audit preparation time versus prior manual processes.

14.1 Supported Compliance Frameworks

Framework	Domain	Automation Potential & Evidence Generated
SOC 2 Type II	Software / Cloud	80%+ automatable — access logs, change records, incidents, availability, backups
ISO 27001	All domains	70%+ automatable — risk assessments, audit logs, policy evidence, change records
GDPR / CCPA	All with personal data	75%+ automatable — PII classification, consent records, access logs, deletion records
SLSA Level 1–4	Software build	99%+ automatable — build attestations, dependency digests, environment definitions
FDA 21 CFR Part 11	Pharma / Medical Devices	50% automatable — electronic signatures, audit trail completeness, data integrity
FDA GMP / cGMP	CPG / Pharma / Food	40% automatable — batch records, equipment calibration, change control records
MiFID II	Financial Services	55% automatable — trade audit trail, best execution records, product governance
Basel III / IV	Banking	45% automatable — risk model documentation, capital calculation audit, stress tests
REACH / EU CSRD	Manufacturing / CPG	40% automatable — substance records, supply chain provenance, sustainability data
FedRAMP / NIST 800-53	Government / Public Sector	50% automatable — security control evidence, scan results, configuration records

14.2 Audit Vault Architecture

The Audit Vault is an immutable, cryptographically hash-chained audit log store. Every state-modifying action in the platform generates an audit event that is written to the Audit Vault. Audit events are:

- Immutable — once written, never modified or deleted
- Cryptographically chained — each event contains a hash of the preceding event
- Retained for 7 years — meeting the most stringent regulatory requirements
- Exportable — streaming export via Kafka, webhook, or S3 batch export for SIEM integration
- Searchable — full-text search via OpenSearch for rapid evidence retrieval

14.3 Compliance Evidence Package Generation

Qala generates compliance evidence packages on demand by querying the Audit Vault, artefact registry, and policy event history. A compliance package for SOC 2 Type II, for example, includes access control reviews, change records, incident records, availability metrics, backup records, and security scan histories — all automatically collated, time-stamped, and signed within minutes.

15 Universal Workflows & Use Cases

Qala's workflows apply coherently across all solution types and domains. A pharmaceutical formulation chemist updating an ingredient specification follows the same governed CCR workflow as a software developer updating a dependency. The workflows are universal; the domain vocabulary adapts.

15.1 Universal Solution Lifecycle Stages

Stage	Description	Scope
1. Initiate	Solution record created; SDE provisioned; owner assigned	All solution types
2. Configure	Configuration versioned; environment definition locked	All solution types
3. Develop	Changes governed by CCR; all artefacts versioned in registry	All solution types
4. Test	Test plans executed; quality gates evaluated; defects tracked	All solution types
5. Review	Approvers sign off; release record assembled; gate checklist complete	All solution types
6. Release	Release event published; distribution channels notified	All solution types
7. Distribute	Distribution records created; consumer notification sent	All solution types
8. Govern	Drift detected; compliance evidence generated; AI recommendations issued	All solution types
9. Retire	Deprecated state set; consumer migration window; archive event	All solution types

15.2 Domain-Specific Workflow Chapters

Domain Chapter	Use Cases Covered
Software & Technology	8 use cases: SDE provisioning, hermetic CI/CD, Blue/Green deployment, security incident quarantine, API breaking change governance, canary release, CVE response, AI-assisted test generation

Domain Chapter	Use Cases Covered
Physical Goods & CPG	7 use cases: New formulation development, ingredient substitution CCR, batch release, product recall, product line change propagation, supply chain sustainability audit, GxP compliance package
Financial Solutions	7 use cases: Financial model SDE provisioning, model validation pipeline, mandate lifecycle management, regulatory examination package, tax rule deployment with effective-date gate, fund inception, model recall
Agricultural & Agribusiness	6 use cases: Field environment SDE, crop lifecycle management, IoT firmware OTA update, harvest batch traceability, seasonal release workflow, farm-to-consumer provenance QR code
Professional Services	6 use cases: Knowledge workspace SDE, methodology version control, consulting engagement governance, service catalogue management, managed service lifecycle, knowledge retrieval
Research & Academic	6 use cases: Research environment SDE, experiment lifecycle, dataset versioning, peer review workflow, publication pipeline, reproducibility verification
Legal, Tax & Regulatory	5 use cases: Legal solution versioning, jurisdiction variant management, tax rule effective-date release, regulatory change CCR, legislative update propagation
Cross-Domain Workflows	6 use cases: Cross-factory artefact sharing, multi-domain solution portfolio governance, AI-generated compliance report, emergency recall across domains, platform marketplace publication, regulatory examination spanning multiple domains

15.3 Key Workflow: Software Release with Blue/Green Deployment

Step	Actor	Action	System Response
1	Tech Lead	Creates Release Record with version, linked artefact digests, linked CCRs	Platform pre-validates SLSA attestations on all linked artefacts
2	Platform	Automated release gate evaluation begins	Test Coverage, SAST, No Critical Defects, Performance, Attestation gates evaluated

Step	Actor	Action	System Response
3	QA Engineer	Reviews gate results, signs off QA approval	Approval recorded with timestamp; compliance evidence created
4	Release Manager	Reviews and approves full release record	Release state → Approved; all approvals collected
5	Platform	Green environment provisioned; new version deployed to Green	Automated smoke tests run against Green; health metrics validated
6	Release Manager	Initiates traffic switch in portal	Load balancer switches 100% traffic from Blue to Green; Blue retained
7	Platform	Post-deployment monitoring active for 30 minutes	AI anomaly detection running; error rate + latency + business metrics watched
8	Release Manager	Confirms release stable after monitoring window	Blue archived; distribution record created; Release state → Live

16 UI/UX Design System

Qala's UI serves pharmaceutical chemists, software engineers, portfolio managers, farm operations managers, tax technologists, and legal directors — all on one platform. The UI feels native to each while remaining one unified system. This is achieved through a persona-adaptive vocabulary layer built on a single shared design system.

16.1 Design Principles

Principle	Description
Universal First, Domain-Native by Persona	Every core concept is universal. Vocabulary, iconography, and shortcuts adapt per persona. A CPG chemist sees "Batch Release Pipeline"; a developer sees "CI/CD Pipeline" — identical underlying system, domain-appropriate surface.
Progressive Disclosure	The most common action for each persona is one click away. Advanced capabilities are discoverable without being visually dominant.
Trust Through Transparency	Every state change is visible. Every action produces an audit event. Users always know: current state, who last changed it, what changed, and what happens next.
AI as Companion, Not Gatekeeper	The AI Agent surfaces recommendations inline — dismissable, actionable, contextual. It never blocks a primary workflow.
Accessibility as Architecture	WCAG 2.1 AA minimum. Every interaction is keyboard-navigable. Screen reader support is a design constraint, not an afterthought.
Speed as a Feature	Data tables load under 200ms. Command Palette (Cmd+K) returns results in < 100ms. Skeleton screens replace spinners.

16.2 Persona-Adaptive Vocabulary

Persona	Primary Nav Items	SDE Vocabulary	Primary Actions
Software Developer	SDEs · Pipelines · Solutions · Tests · Releases · Artefacts	SDE	Commit, build, test, release, deploy
CPG / Pharma Chemist	Workbenches · Formulations · Batches · QA Records · Releases	Formulation Workbench	Formulate, simulate, test, batch release

Persona	Primary Nav Items	SDE Vocabulary	Primary Actions
Portfolio Manager	Research Workbenches · Funds · Models · Mandates · Compliance	Research Workbench	Backtest, risk-gate, mandate, activate
Farm Ops Manager	Field Environments · Crops · Devices · Yields · Seasons	Field Environment	Monitor, irrigate, harvest, trace
Principal Consultant	Knowledge Workspaces · Methodologies · Engagements · Catalogue	Knowledge Workspace	Develop methodology, engage, deliver, publish
Research Scientist	Research Environments · Experiments · Datasets · Publications	Research Environment	Experiment, analyse, peer-review, publish
Tax Product Manager	Tax Workspaces · Rule Sets · Jurisdictions · Legislative Feed	Tax Workspace	Draft rule set, validate, schedule, activate
Compliance Officer	Policies · Compliance Reports · CCR Queue · Audit Trail	N/A (no SDE)	Audit, evidence, report, certify

16.3 Brand Colour Palette

Token	Usage
Brand Navy (#0D2B45)	Primary headings, nav background, logo, brand anchors — WCAG AAA 18.1:1
Action Blue (#1D6FA4)	Interactive elements, links, primary buttons, progress indicators — WCAG AA 5.9:1
Sky Light (#D6EAF8)	Tints, info panels, selected state backgrounds, focus rings
Surface White (#FFFFFF)	Primary content surface, card background, dialog background
Surface Grey (#F4F6F8)	Alternate rows, secondary surface, sidebar track, divider fill
Text Primary (#1A1A2E)	Body text, labels, headings on white surface — WCAG AAA 18.1:1
AI Purple (#4A235A)	AI Agent indicator, recommendation badge, intelligence features — WCAG AA 11.3:1
Success Dark (#1A5C38)	Gate passed, approved, active, deployed states — WCAG AA 10.1:1
Danger Dark (#7B241C)	Error state, critical alert, failed gate, recall — WCAG AA 9.1:1

17 Deployment Models & Infrastructure

17.1 Deployment Models

Model	Description	Best For
Qala SaaS (Multi-tenant)	Managed by Qala. Shared infrastructure. Per-factory isolation via encryption and namespace separation.	Individuals, SMBs, most enterprises
Qala SaaS (Single-tenant)	Managed by Qala. Dedicated infrastructure for one customer. Full isolation.	Enterprises with strict data isolation requirements
Self-Hosted (Cloud)	Customer installs Qala on their own cloud (AWS, Azure, GCP). Qala provides Helm chart and operator.	Enterprises requiring data sovereignty or custom cloud configuration
Self-Hosted (On-Premises)	Customer installs Qala on their own data centre infrastructure.	Regulated industries with on-premises mandates (defence, government, healthcare)
Air-Gapped	Fully isolated installation with no external network access. Manual update packages.	Classified environments, critical infrastructure, maximum security
Edge / Embedded	Lightweight Qala node deployed to edge devices (IoT gateways, field equipment).	Agricultural IoT, point-of-sale, remote lab or field operations

17.2 Non-Functional Requirements

Metric	Target
API P99 Latency (read)	< 80ms under nominal load (1,000 RPS/tenant)
API P99 Latency (write)	< 200ms for command acceptance (event published)
Platform Availability	99.9% SaaS Standard; 99.95% SaaS Enterprise
Event Processing Throughput	> 100,000 domain events/sec platform-wide
Storage Durability	99.999999999% (11 nines) for artefact storage
Max SDE Boot Time	< 30 seconds from activation to first API response
Tenant Onboarding	< 2 minutes from signup to first usable SDE

Metric	Target
Recovery Point Objective (RPO)	< 1 minute (event log replication lag target)
Recovery Time Objective (RTO)	< 15 minutes (full platform recovery)
Security Scan SLA (Critical)	Critical CVEs patched within 24 hours
Security Scan SLA (High)	High CVEs patched within 72 hours
Hermetic Build Success Rate	> 99% on first run
Backup Success Rate	> 99.9% of scheduled backups succeed

17.3 Repository Structure

Path	Contents
contracts/events/	Shared event schemas (JSON Schema) — single event contract for all services
contracts/openapi/	OpenAPI specifications for all REST endpoints
db/postgres/	SQL schema definitions and migration scripts; one schema file per service
docs/	Architecture documentation, SDD traceability matrix, systems-subsystems reference
go/	Control Plane microservices (Go): gateway, identity, SDE, workspace, artefacts
rust/	Kernel System and Intelligence & Security Plane: qala-kernel, ai-agents, security-sem, qala-shared
scala/	Execution Plane services: workflow-ci-cd, data-platform, notifications
infra/	Docker Compose (local dev), Kubernetes manifests, Terraform configurations

18 Business Model & Market Position

18.1 Market Position

Qala creates and dominates a new software category: Universal Solution Lifecycle Management. No direct competitor serves all solution types on one governed platform. The platform addresses six converging markets with a total addressable market of over \$163 billion.

Market Segment	2024 Size	CAGR	Qala Relevance
Enterprise DevOps / ALM	\$18.4B	14.2%	Software SDE, CI/CD pipeline, artefact registry
Quality Management Systems (QMS)	\$14.7B	10.8%	CPG, pharma quality gates + CCR
Enterprise Risk & Compliance	\$56.3B	9.4%	Cross-domain CCR, risk scoring, audit trail
Supply Chain Traceability	\$22.1B	18.7%	CPG/agri traceability, GS1 EPCIS, distribution records
Financial Risk & Portfolio Mgmt	\$34.2B	8.6%	Financial SDE, model governance, mandate lifecycle
AI / ML Ops Platforms	\$4.1B	38.2%	AI Agent, cross-domain intelligence, model registry

18.2 Pricing Model

Tier	Target	Pricing Structure	Annual Range
Starter	Growth companies, single domain	\$150/user/month + \$5K base; up to 50 users; 1 domain	\$10K – \$90K/year
Professional	Mid-market, 2–3 domains	\$800/domain/month + \$250/user/month	\$80K – \$320K/year
Enterprise	Large enterprises, 4+ domains	Annual contract; platform fee + domain fee + user fee + success package	\$320K – \$2M+/year
Public Sector	Government, defence, regulatory agencies	Annual contract; government pricing schedule; sovereign cloud option	\$120K – \$1.2M/year

18.3 5-Year Financial Projections

Metric	Y1 2025	Y2 2026	Y3 2027	Y4 2028	Y5 2029
Total Revenue	\$10.0M	\$28.0M	\$66.3M	\$122.7M	\$203.7M
Gross Margin %	68%	76%	77%	80%	84%
EBITDA	(\$12.6M)	(\$15.3M)	(\$10.4M)	\$11.5M	\$60.1M
EBITDA Margin	(126%)	(55%)	(16%)	9%	30%

Path to Profitability

EBITDA positive by Q3 2028. Unit economics improve as: (1) sales efficiency matures post Series A, (2) domain expansion revenue (90% gross margin) grows as % of ARR, (3) AI Agent reduces customer success headcount per logo, (4) infrastructure costs flatten while ARR scales.

19 Risks, Constraints & Open Design Questions

19.1 Platform Risks

Risk	Likelihood	Impact	Mitigation
Toolchain complexity overwhelms new users	High	High	Opinionated defaults, guided onboarding, curated templates for common use cases
AI recommendations of poor quality erode trust	Medium	High	Track acceptance rates; require human review for AI-generated actions; invest in feedback loops
Platform lock-in perception deters adoption	Medium	High	Open standards for all data formats; import/export for all entities; never use proprietary formats
Performance at scale exceeds architecture limits	Low	Critical	Architect for horizontal scaling from day one; load-test at 10× expected peak before launch
Regulatory changes break compliance modules	Medium	Medium	Modular compliance framework; rapid update cycle; regulatory monitoring capability
Supply chain attack on Qala platform itself	Low	Critical	Platform built with Qala hermetic builds and SLSA attestations — eat our own cooking
Fragmented adoption (some teams, not all)	High	Medium	Factory-level enforcement makes partial adoption self-defeating; leadership sponsorship strategy

19.2 Key Open Design Questions (RFC v2.0)

The following architectural questions remain open for decision. Each is formally tracked as an RFC with a decision deadline:

Q-ARCH-001	Is Scala/Akka reactive streaming worth the language diversity cost vs. Go + Temporal?	Pre-alpha build
Q-ARCH-002	For the Kernel Plane, is Rust memory safety genuinely necessary vs. Go?	Pre-alpha build
Q-SDE-001	Should "SDE" be unified with modular capability packs or explicitly subtyped per domain?	Pre-alpha

Q-SDE-004	Should non-software SDE types be in scope for v1 or phased via Domain Packs?	Pre-alpha
Q-HB-002	For stochastic processes (ML training), should attestation require "governed reproducibility" vs. exact?	Pre-beta
Q-EVT-001	Should AUDIT_EVENTS write to a dedicated append-only hash-chained store separate from Kafka?	Pre-alpha build
Q-AI-001	Should the AI Agent use a single universal model or domain-specific models?	Pre-beta
Q-AI-002	How should the AI Agent produce explainable outputs for regulated domain solutions (EU AI Act)?	Pre-GA
Q-MODEL-001	Should solution types be a strict flat list or a hierarchy (categories > types > subtypes)?	Pre-alpha
Q-COMP-001	Should Qala build its own compliance evidence engine or partner with a GRC platform?	Pre-GA

20 Implementation Traceability Matrix

This matrix maps every SDD capability to its concrete implementation in the Qala codebase, providing full traceability from design specification to code.

SDD Capability	Implemented In
API Gateway routing (/api/*)	go/cmd/api-gateway/main.go
User CRUD + USER_EVENTS	go/cmd/user-identity-service/main.go
SDE lifecycle + snapshot/rollback + SDE_EVENTS	go/cmd/sde-management-service/main.go
Solution hierarchy + solution factory orchestration	go/cmd/sde-management-service/main.go
Workspace CRUD + content operations + CMS_EVENTS	go/cmd/workspace-cms-service/main.go
Pipeline run/status + BUILD_EVENTS	scala/workflow-ci-cd-service/src/main/scala/com/qala/workflow/Main.scala
Artifact upload/download + ARTIFACT_EVENTS	go/cmd/artifact-package-service/main.go
Data pipeline + analytics + DATA_EVENTS	scala/data-platform-service/src/main/scala/com/qala/data/Main.scala
AI recommendations + SDE analysis + AI_RECOMMENDATIONS	rust/ai-agents-service/src/main.rs
Security threats/policies/scan + SECURITY_EVENTS	rust/security-sem-service/src/main.rs
Notification dispatch + NOTIFICATIONS	scala/notifications-service/src/main/scala/com/qala/notifications/Main.scala
Kernel OS coordination (systems/subsystems/events/commands)	rust/qala-kernel/src/main.rs, rust/qala-kernel/src/state.rs
Shared event contract	contracts/events/schema.json, rust/qala-shared/src/lib.rs, go/internal/platform/events/events.go
DB schema blueprint	db/postgres/*.sql

20.1 Systems and Subsystems Reference

Subsystem	Responsibility
Kernel System: Registry	Tracks registered services and heartbeat status

Subsystem	Responsibility
Kernel System: CommandBus	Executes kernel commands (set_subsystem_status, set_service_status)
Kernel System: EventAggregator	Counts and stores recent topic events
Control Plane: Gateway	Ingress and routing to microservices
Control Plane: Identity	Users and RBAC metadata
Control Plane: SDE	SDE lifecycle, solution modeling, solution factory coordination
Control Plane: Workspace	Workspace and file content management
Control Plane: Artifact	Package and binary metadata
Execution Plane: Workflow	CI/CD pipeline trigger and status
Execution Plane: Data	Metrics and analytics computation
Execution Plane: Notifications	User/system notification dispatch
Intelligence Plane: AI	Recommendation and analysis hooks
Security Plane: SEM	Security events, policy updates, SDE scanning and quarantine decisions

21 Glossary

Term	Definition
AI Agent	Qala's persistent, context-aware intelligence layer that operates continuously across a customer's solution estate
Artefact	A produced output: compiled binary, package, container image, specification, test result, or release bundle
Audit Vault	Immutable, cryptographically hash-chained audit log store with 7-year retention
CCR	Change Control Request — the primary governance instrument for all intentional modifications to any entity
CCB	Change Control Board — the approval body for Critical and High-risk CCRs
CMS	Content Management System — manages all structured and unstructured content within an SDE
Domain Pack	A versioned JSON Schema extension that adds domain-specific fields, validation rules, and quality gates to the Universal Solution Schema
Factory Blueprint	A packaged, versioned, shareable Solution Factory configuration publishable to the Qala Marketplace
Hermetic Build	A build that executes in complete isolation with digest-pinned dependencies, producing bit-for-bit identical outputs from identical inputs
Kernel	The Rust-based low-level coordination layer: service registry, health monitoring, command bus, event aggregation
Playbook	The blueprint and design document for a solution — captures design decisions, architectural choices, and implementation patterns
RLS	Row-Level Security — PostgreSQL mechanism enforcing tenant isolation at the database layer
SDE	Solution Development Environment — the foundational deployable, configurable, distributable workspace for developing solutions
SEM	Security Event Manager — subsystem providing continuous security monitoring, threat detection, policy enforcement, and SDE quarantine

Term	Definition
SLSA	Supply-chain Levels for Software Artefacts — open standard for build attestation and supply chain integrity
Solution	A concrete, versioned instance of a Solution Model — the actual thing being built, managed, and delivered
Solution Book	The comprehensive documentation and content repository for a solution — from inception through retirement
Solution Factory	A governed namespace that produces and manages Solution Development Environments, child factories, and all solution activity within a defined scope
Solution Model	The structured definition, schema, and behavioral specification governing how a class of solutions is created and managed
Universal Solution Schema	The common core schema shared by all solution types; extended by Domain Packs